

RESTful Web Services in Spring Boot

In diesem Modul

- REST-Controller Basics: ResponseEntity, Content Negotiation
- Validation & Error Handling
- Moderne HTTP Clients: RestClient, Declarative HTTP Interfaces
- Async/Streaming: CompletableFuture, Server-Sent Events
- API-Dokumentation, Contracts & Versionierung
- Demo und Übungsaufgabe

Wiederholung: Der @RestController

- Spezielle `Controller`-Annotation, die `Controller` und `@ResponseBody` kombiniert.
- Jede Methode gibt direkt Daten zurück (keine View-Auflösung).
- Behandelt JSON/XML-Serialisierung automatisch.

```
@RestController
@RequestMapping("/api/v1/users")
public class UserController {

    @GetMapping("/{id}")
    public User getUserById(@PathVariable Long id) {
        // ... Logik
        return new User(id, "Alice");
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED) // Setzt den HTTP Status 201
    public User createUser(@RequestBody User user) {
        // ... Logik
        return user;
    }
}
```

Wiederholung: ResponseEntity

Volle Kontrolle über HTTP Response (Status, Header, Body).

```
@GetMapping("/{id}")  
public ResponseEntity<User> findUser(@PathVariable Long id) {  
    Optional<User> user = userService.findById(id);  
    return user.map(ResponseEntity::ok) // 200 OK  
               .orElse(ResponseEntity.notFound().build()); // 404 Not Found  
}
```

Content Negotiation

Client und Server einigen sich auf das beste Datenformat.

- **Accept Header (Client):** `Accept: application/json` oder `Accept: application/xml`
- **Content-Type Header (Server):** `Content-Type: application/json`
- Spring Boot nutzt `HttpMessageConverter` um automatisch zu serialisieren/deserialisieren. Standard ist JSON (Jackson).

Content Negotiation: Beispiel

```
// Beispiel: XML über Jackson-Converter
// Dependency: 'com.fasterxml.jackson.dataformat:jackson-dataformat-xml'
@RestController
public class ProductController {
    @GetMapping(value = "/products/{id}", produces = MediaType.APPLICATION_XML_VALUE)
    public Product getProductXml(@PathVariable Long id) {
        return new Product(id, "Laptop");
    }
}
```

DTO-Validierung

Das Bean Validation-Framework wird auch vom Spring Framework unterstützt, um Daten mit verschiedenen Annotationen zu prüfen.

Die wichtigsten Annotationen:

- `@Valid` : Standard-JSR 380 (Bean Validation) Annotation.
- `@Validated` : Spring-spezifisch, unterstützt Validation Groups.

DTO-Validierung

```
// DTO für die Anfrage
public class UserCreatedto {
    @NotBlank(message = "Name darf nicht leer sein")
    @Size(min = 3, max = 50, message = "Name muss 3-50 Zeichen haben")
    private String name;

    @Email(message = "Ungültiges E-Mail Format")
    private String email;

    // Getter & Setter
}

@PostMapping
public ResponseEntity<User> createUser(@Valid @RequestBody UserCreatedto userDto) {
    // Wenn Validation fehlschlägt, wird eine MethodArgumentNotValidException geworfen
    // ...
}
```

Validation Groups

Unterschiedliche Regeln für verschiedene Szenarien (z.B. Erstellen vs. Aktualisieren).

```
// Interfaces als Marker
public interface OnCreate {}
public interface OnUpdate {}

public class UserDto {
    @NotNull(groups = OnUpdate.class) // Nur bei Update nötig
    private Long id;

    @NotBlank(groups = OnCreate.class) // Nur bei Create nötig
    private String name;
}

@PostMapping
public ResponseEntity<User> create(@Validated(OnCreate.class) @RequestBody UserDto dto) { ... }

@PutMapping
public ResponseEntity<User> update(@Validated(OnUpdate.class) @RequestBody UserDto dto) { ... }
```

Custom Validators

Eigene Validierungslogik implementieren.

```
@Target({ElementType.FIELD})
@Retention(RetentionPolicy.RUNTIME)
@Constraint(validatedBy = UniqueEmailValidator.class)
public @interface UniqueEmail {
    String message() default "E-Mail bereits vergeben";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}

public class UniqueEmailValidator implements ConstraintValidator<UniqueEmail, String> {
    @Autowired private UserRepository userRepository;

    @Override
    public boolean isValid(String email, ConstraintValidatorContext context) {
        return userRepository.findByEmail(email).isEmpty();
    }
}
```

Globales Error Handling

Zentrales Fehlerhandling für die gesamte REST-API.

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ProblemDetail handleValidationExceptions(MethodArgumentNotValidException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.BAD_REQUEST, "Validierungsfehler");
        pd.setTitle("Invalid Request Body");
        return pd;
    }

    @ExceptionHandler(ResourceNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ProblemDetail handleNotFound(ResourceNotFoundException ex) {
        return ProblemDetail.forStatusAndDetail(
            HttpStatus.NOT_FOUND, ex.getMessage());
    }
}
```

ProblemDetails (RFC 7807)

Standardisiertes Format für HTTP API Fehlerantworten (seit Spring Boot 3).

```
{
  "type": "about:blank",
  "title": "Invalid Request Body",
  "status": 400,
  "detail": "Validierungsfehler",
  "instance": "/api/v1/users",
  "errors": {
    "name": "Name darf nicht leer sein",
    "email": "Ungültiges E-Mail Format"
  }
}
```

Spring Boot konvertiert `ProblemDetail` automatisch in JSON oder XML, wenn der `Accept` -Header dies verlangt.

ProblemDetails erweitern

Oft reicht der Standard nicht. Wir wollen z.B. eine `traceId` oder spezifische Business-Error-Codes hinzufügen.

```
@ExceptionHandler(MyBusinessException.class)
public ProblemDetail handleBusinessException(MyBusinessException ex) {
    ProblemDetail pd = ProblemDetail.forStatusAndDetail(
        HttpStatus.BAD_REQUEST, ex.getMessage());

    // Eigene Properties hinzufügen
    pd.setProperty("errorCode", "BUS-001");
    // Angenommen wir haben einen Tracer injectet
    pd.setProperty("traceId", tracer.currentSpan().context().traceId());

    return pd;
}
```

Moderne HTTP Clients

Status Quo: RestTemplate

- Lange Zeit der Standard für synchrone Calls.
- Jetzt im **Maintenance Mode**. Es wird keine neuen Features mehr geben.
- Nachteil: Viele überladene Methoden, kein Fluent API.

Die Nachfolger:

1. **RestClient (Spring Boot 3.2)**: Synchron, Fluent API. Basiert auf Servlet-Stack.
2. **WebClient (Spring 5)**: Reaktiv, non-blocking. Erfordert `spring-boot-starter-webflux`.
3. **Declarative HTTP Interfaces (Spring 6)**: Interface-basiert (via Proxy).

Der RestClient (Synchron)

Bietet eine moderne Fluent API ohne Reactive Stack (Mono/Flux).

```
@Service
public class ProductClient {
    private final RestClient restClient;

    public ProductClient(RestClient.Builder builder) {
        this.restClient = builder.baseUrl("https://api.example.com").build();
    }

    public Product getProduct(String id) {
        return restClient.get()
            .uri("/products/{id}", id)
            .retrieve()
            .body(Product.class);
    }
}
```

Declarative HTTP Interfaces

Definiere die API als Java Interface (ähnlich Feign/Retrofit).

```
public interface UserApi {  
    @GetExchange("/users/{id}")  
    User getById(@PathVariable Long id);  
  
    @PostExchange("/users")  
    void createUser(@RequestBody User user);  
}
```

Dies benötigt einen **Unterbau**, der die Requests ausführt (WebClient oder RestClient).

Declarative Client Factory (mit RestClient)

Verbindung von Interface und Engine.

```
@Configuration
public class ClientConfig {
    @Bean
    UserApi userApi(RestClient.Builder builder) {
        RestClient client = builder.baseUrl("https://user-service").build();

        // Nutzt den synchronen RestClient als Engine
        RestClientAdapter adapter = RestClientAdapter.create(client);

        HttpServiceProxyFactory factory = HttpServiceProxyFactory
            .builderFor(adapter)
            .build();

        return factory.createClient(UserApi.class);
    }
}
```

Asynchrone APIs & Streaming

CompletableFuture als Rückgabotyp

- Der Controller Thread wird freigegeben, während die Logik im Hintergrund arbeitet.
- Verbessert die Skalierbarkeit bei blockierenden Operationen.

```
@RestController
public class AsyncController {
    @Autowired private SlowService slowService;

    @GetMapping("/async-result")
    public CompletableFuture<String> getAsyncResult() {
        return CompletableFuture.supplyAsync(() -> slowService.doSlowWork());
    }
}
```

Streaming Responses (Server-Sent Events)

Für Realtime-Updates, z.B. wenn der Client ständig neue Daten erhalten soll.

```
@RestController
public class SseController {

    @GetMapping(value = "/events", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    public SseEmitter streamEvents() {
        SseEmitter emitter = new SseEmitter();

        new Thread(() -> {
            // Hier: Events asynchron an den Emitter senden
            // z.B. aus einem Message Queue Listener oder einem Scheduled Task
            emitter.send(SseEmitter.event().name("message").data("Hello, Client!"));
            emitter.complete(); // Verbindung schließen
        }).start();

        return emitter;
    }
}
```

OpenAPI (früher Swagger)

Ziel

Standardisierte, maschinenlesbare Beschreibung von REST-APIs.

- **Dokumentation:** Interaktive UI (Swagger UI).
- **Code-Generierung:** Clients in jeder Sprache.

Man unterscheidet zwei Ansätze: Code First und Contract First.

Ansatz 1: Code-First (SpringDoc OpenAPI)

Man schreibt den Code, die Doku wird daraus generiert.

```
@RestController
@RequestMapping("/products")
@Tag(name = "Produktverwaltung", description = "API für CRUD-Operationen an Produkten")
public class ProductController {

    @Operation(
        summary = "Produkt anhand ID abrufen",
        description = "Gibt ein Produktobjekt basierend auf der bereitgestellten ID zurück.",
        parameters = @Parameter(name = "id", description = "ID des Produkts", example = "123")
    )
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Produkt gefunden",
            content = @Content(schema = @Schema(implementation = Product.class))),
        @ApiResponse(responseCode = "404", description = "Produkt nicht gefunden")
    })
    @GetMapping("/{id}")
    public Product getProduct(@PathVariable Long id) {
        return new Product(id, "Advanced Widget");
    }
}
```

Ansatz 2: Contract-First

Man schreibt zuerst die OpenAPI-Spezifikation (YAML/JSON) und generiert daraus den Code (Interfaces, DTOs).

Vorteile:

- **API-Design als erste Klasse:** Fokus auf das API-Design, bevor implementiert wird.
- **Parallele Entwicklung:** Backend- und Frontend-Teams können gleichzeitig arbeiten.
- **Konsistenz:** API ist über alle Services hinweg konsistent.

Tool: `openapi-generator-maven-plugin` (oder Gradle Plugin).

HATEOAS (Hypermedia As The Engine Of Application State)

Idee

API-Clients navigieren durch die Anwendung mittels Links in den Responses, anstatt hartkodierte URLs zu verwenden.

Macht APIs flexibler und selbst-beschreibender.

Dependency: `spring-boot-starter-hateoas`

```
// Ein DTO, das Links enthalten kann
public class OrderModel extends RepresentationModel<OrderModel> {
    private String orderId;
    private String status;
    // ... andere Felder
}

@RestController
@RequestMapping("/orders")
public class OrderController {

    @GetMapping("/{orderId}")
    public EntityModel<Order> getOrder(@PathVariable String orderId) {
        Order order = new Order(orderId, "PENDING");

        // Füge Links hinzu
        return EntityModel.of(order,
            linkTo(methodOn(OrderController.class).getOrder(orderId)).withSelfRel(),
            linkTo(methodOn(OrderController.class).cancelOrder(orderId)).withRel("cancel")
        );
    }

    @PostMapping("/{orderId}/cancel")
    public ResponseEntity<?> cancelOrder(@PathVariable String orderId) {
        // ... Logik
        return ResponseEntity.noContent().build();
    }
}
```

Beispiel Response (HAL JSON):

```
{
  "orderId": "ORD123",
  "status": "PENDING",
  "_links": {
    "self": { "href": "http://localhost:8080/orders/ORD123" },
    "cancel": { "href": "http://localhost:8080/orders/ORD123/cancel" }
  }
}
```

Exkurs: REST vs. GraphQL

Bei komplexen Datenmodellen stößt REST an Grenzen (**Overfetching** / **Underfetching**).

- **REST:** Server definiert die Struktur der Antwort.
 - *Gut für:* Caching, Simple APIs, Public APIs.
- **GraphQL:** Client definiert die Struktur der Antwort (Query).
 - *Gut für:* Mobile Apps, komplexe Frontends, Aggregation.

Spring for GraphQL: Baut auf `Controller` -Logik auf (`@QueryMapping` , `@SchemaMapping`) und ist mittlerweile erstklassig integriert.

API Versionierung

Warum Versionierung?

- APIs entwickeln sich weiter, Clients müssen nicht gleichzeitig updaten.
- Abwärtskompatibilität ist entscheidend.

Es gibt hauptsächlich drei Strategien: URI-Versionierung, Header-Versionierung und Media Type-Versionierung.

Strategien

1. URI Versionierung (Path Versioning)

Die einfachste und gängigste Methode.

```
@RestController
@RequestMapping("/api/v1/users")
public class UserControllerV1 { /* ... */ }

@RestController
@RequestMapping("/api/v2/users")
public class UserControllerV2 { /* ... */ }
```

Vorteil: Klar im URL, einfach zu routen.

Nachteil: Verletzt HATEOAS, da Links sich ändern müssten.

2. Header Versionierung

Die Version wird im HTTP Header gesendet.

`X-API-VERSION: 1` oder `X-API-VERSION: 2`

Vorteil: Hält den URI sauber, HATEOAS-freundlich.

Nachteil: Nicht so gut sichtbar, Clients müssen den Header kennen.

```
@GetMapping(value = "/users/{id}", headers = "X-API-VERSION=1")  
public User getUserV1(@PathVariable Long id) { /* ... */ }
```

3. Media Type Versionierung (Accept Header)

Die Version ist Teil des `Accept`-Headers.

```
Accept: application/vnd.company.v1+json
```

Vorteil: Sehr RESTful, HATEOAS-freundlich.

Nachteil: Komplexere Implementierung.

```
@GetMapping(value = "/users/{id}", produces = "application/vnd.company.v1+json")  
public User getUserV1(@PathVariable Long id) { /* ... */ }
```